Lab Goal : This lab was designed to teach you how to solve a weighted shortest path problem using Dijkstra's algorithm on an adjacency list.

Lab Description : The data file contains the Dijkstra's problems from the first Dijkstra's worksheet, so you can check your results against it. There are also two undirected graphs in the data file. All graphs are shown below. You will find all of the shortest paths from vertices given in the data file.

algorithm help

Before the loop,

- make maps to store the costs and previous nodes and a set to keep track of the visited nodes.
- Map each vertex to a cost of Double.POSITIVE_INFINITY and previous as null.
- Set the cost of the starting node to zero.

Loop size times:

- Find the node with the smallest cost not yet visited and set is as the current node
- If there are no more reachable nodes, stop looping.
- Mark the current node as visited
- Loop thru all neighbors of the current node:
  - If the neighbor has already been visited:
    - Skip it
  - Calculate the tentative cost this neighbor
  - If the tentative cost is cheaper than the current cost stored for this neighbor:
    - Update the cheapest cost and prev

Sample Graphs and Outputs*:

Graph #1 - Directed Graph

```
{A=[C2.0, D8.0, E3.0],
 B=[D6.0],
 C=[B3.0, D4.0, E9.0],
 D=[],
 E=[],
 F=[A1.0]}
```

Dijkstra's Algorithm starting at F:

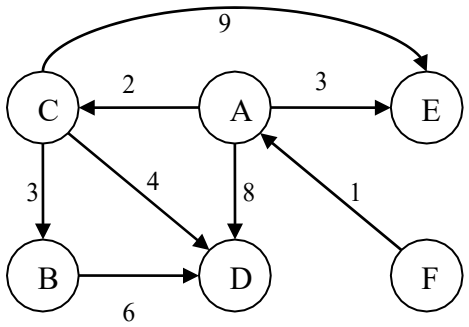
```
Minimum Distances:
A     B     C     D     E     F
1.0   6.0   3.0   7.0   4.0   0.0
F     C     A     C     A     null

Path from F to A:  F -> A
Path from F to B:  F -> A -> C -> B
Path from F to C:  F -> A -> C
Path from F to D:  F -> A -> C -> D
Path from F to E:  F -> A -> E
Path from F to F:  F
```

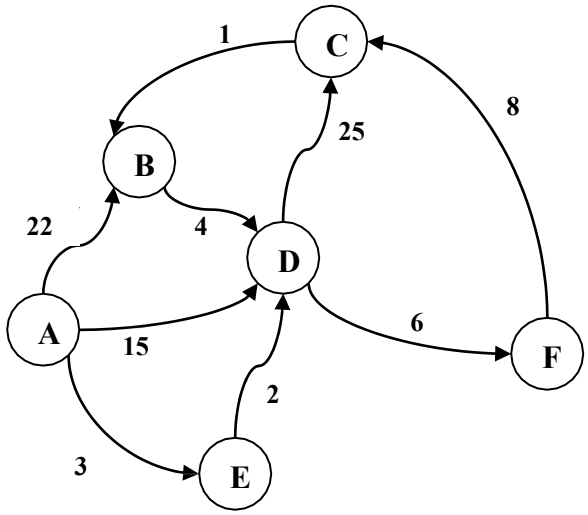
Dijkstra's Algorithm starting at A:

```
Minimum Distances:
A     B     C     D     E     F
0.0   5.0   2.0   6.0   3.0   inf
null  C     A     C     A     null

Path from A to A:  A
Path from A to B:  A -> C -> B
Path from A to C:  A -> C
Path from A to D:  A -> C -> D
Path from A to E:  A -> E
Path from A to F:  does not exist.
```



* There are two text files provided called expected1.out and expected2.out. The first has the same outputs as shown here. The second shows the state of the costs, predecessors, and visited structures after every iteration of dijkstra's. Move the code the shows the state in or out of the dijkstra's loop to control how much is outputted.



Graph #2 - Directed Graph

```
{A=[B22.0, D15.0, E3.0],
 B=[D4.0],
 C=[B1.0],
 D=[F6.0, C25.0],
 E=[D2.0],
 F=[C8.0]}
```

Dijkstra's Algorithm starting at A:

```
Minimum Distances:
A     B     C     D     E     F
0.0   20.0  19.0  5.0   3.0   11.0
null  C     F     E     A     D

Path from A to A:  A
Path from A to B:  A -> E -> D -> F -> C -> B
Path from A to C:  A -> E -> D -> F -> C
Path from A to D:  A -> E -> D
Path from A to E:  A -> E
Path from A to F:  A -> E -> D -> F
```

Dijkstra's Algorithm starting at B:

```
Minimum Distances:
A     B     C     D     E     F
inf   0.0   18.0  4.0   inf   10.0
null  null  F     B     null  D

Path from B to A:  does not exist.
Path from B to B:  B
Path from B to C:  B -> D -> F -> C
Path from B to D:  B -> D
Path from B to E:  does not exist.
Path from B to F:  B -> D -> F
```

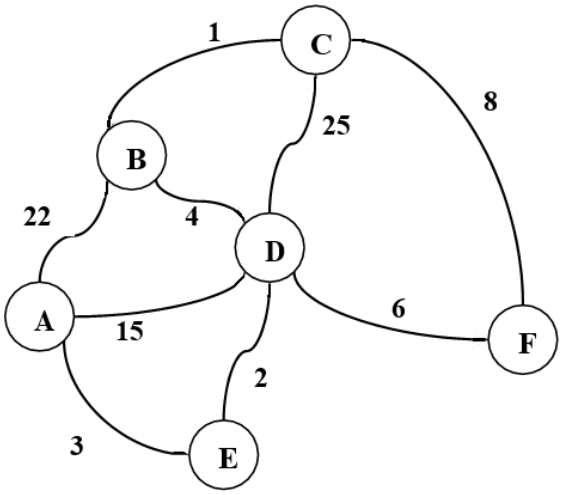
Graph #3 - Undirected Graph

```
{A=[B22.0, D15.0, E3.0],
 B=[A22.0, D4.0, C1.0],
 C=[B1.0, D25.0, F8.0],
 D=[A15.0, B4.0, F6.0, C25.0, E2.0],
 E=[A3.0, D2.0],
 F=[D6.0, C8.0]}
```

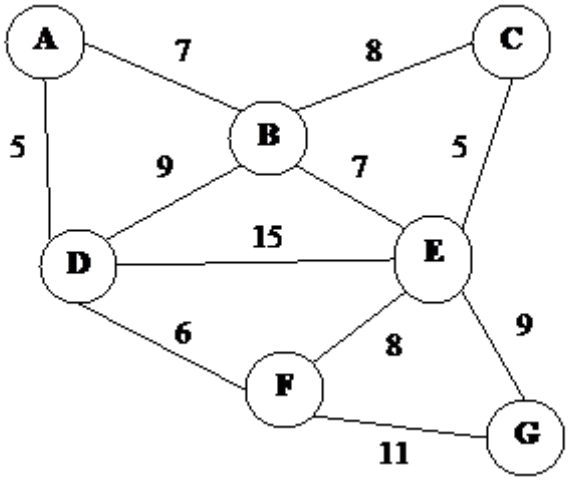
Dijkstra's Algorithm starting at E:

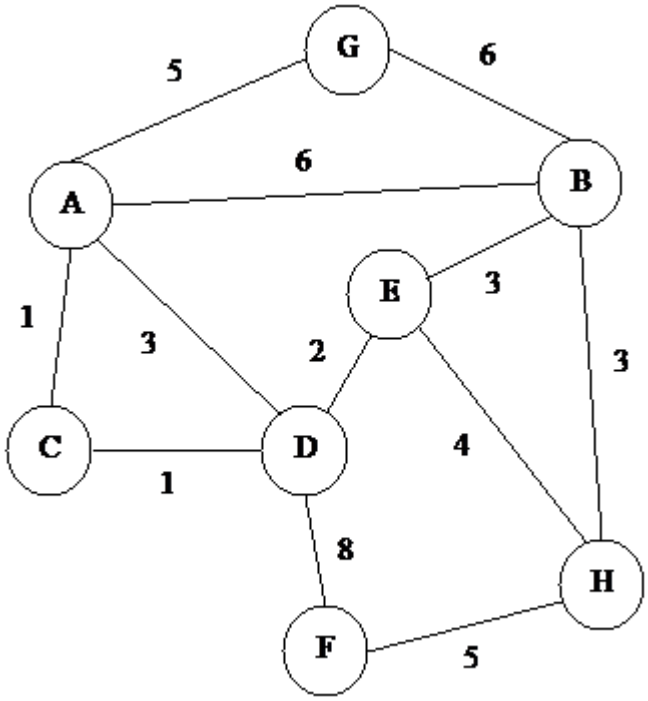
```
Minimum Distances:
A     B     C     D     E     F
3.0   6.0   7.0   2.0   0.0   8.0
E     D     B     E     null  D

Path from E to A:  E -> A
Path from E to B:  E -> D -> B
Path from E to C:  E -> D -> B -> C
Path from E to D:  E -> D
Path from E to E:  E
Path from E to F:  E -> D -> F
```



Sample Data :





Graph #4 - Undirected Graph

```
{A=[B7.0, D5.0],
 B=[A7.0, C8.0, D9.0, E7.0],
 C=[B8.0, E5.0],
 D=[A5.0, B9.0, E15.0, F6.0],
 E=[B7.0, C5.0, D15.0, F8.0, G9.0],
 F=[D6.0, E8.0, G11.0],
 G=[E9.0, F11.0]}
```

Dijkstra's Algorithm starting at A:

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	22.0
null	A	B	A	B	D	F

```
Path from A to A:  A
Path from A to B:  A -> B
Path from A to C:  A -> B -> C
Path from A to D:  A -> D
Path from A to E:  A -> B -> E
Path from A to F:  A -> D -> F
Path from A to G:  A -> D -> F -> G
```

Graph #5 - Undirected Graph

```
{A=[B6.0, C1.0, D3.0, G5.0],
 B=[A6.0, E3.0, G6.0, H3.0],
 C=[A1.0, D1.0],
 D=[A3.0, C1.0, E2.0, F8.0],
 E=[B3.0, D2.0, H4.0],
 F=[D8.0, H5.0], G=[A5.0, B6.0],
 H=[B3.0, E4.0, F5.0]}
```

Dijkstra's Algorithm starting at A:

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

```
Path from A to A:  A
Path from A to B:  A -> B
Path from A to C:  A -> C
Path from A to D:  A -> C -> D
Path from A to E:  A -> C -> D -> E
Path from A to F:  A -> C -> D -> F
Path from A to G:  A -> G
Path from A to H:  A -> C -> D -> E -> H
```